GLA UNIVERSITY MATHURA

Accredited with A Grade by NAAC

12-B Status from UGC

Programming in Java (BCAC 0023)

Module-1

Faculty Name –

Mr. Shivank Chauhan

Class

- A class is a user defined **blueprint** or prototype from which **objects** are created.
- It represents the *set of properties* or **methods** that are common to all objects of one type.
- In terms of object-oriented programming, a class is like a template from which an **instance** of that class is created at run time Behavior

Syntax to declare class

```
class Class-name
 {
        properties  // or data members or variables
        methods // functions or Behavior
 }
```

Here class-name is any valid identifier name.

Methods define the **behavior (functions)** of the objects.

Properties defines the objects **states(values).**

```java
// A sample program to illustrate classes in Java
package module2;
class Circle
{
    double radius,area;
    void set_radius(double r){
        radius = r;
    }
    void display_circle_area(){
        area = 3.14*radius*radius;
        System.out.print("\n Radius =" + radius);
        System.out.println("\n Area =" + area);
    }
}
public class Classexample1{
    public static void main(String args[]){
        Circle c1 = new Circle();
        c1.set_radius(2.0);
        c1.display_circle_area();
    }
}
```

Defining Class

- We can define a class using the keyword class and its class name.

- For Example

```
class Circle
{



}
```

Adding Variables

- We can add variables by specifying the data type and name of variables
- For example

```
class Circle
{
    double radius,area;

}
```

Here double is the data type and radius, area are variables of the class

Adding Methods

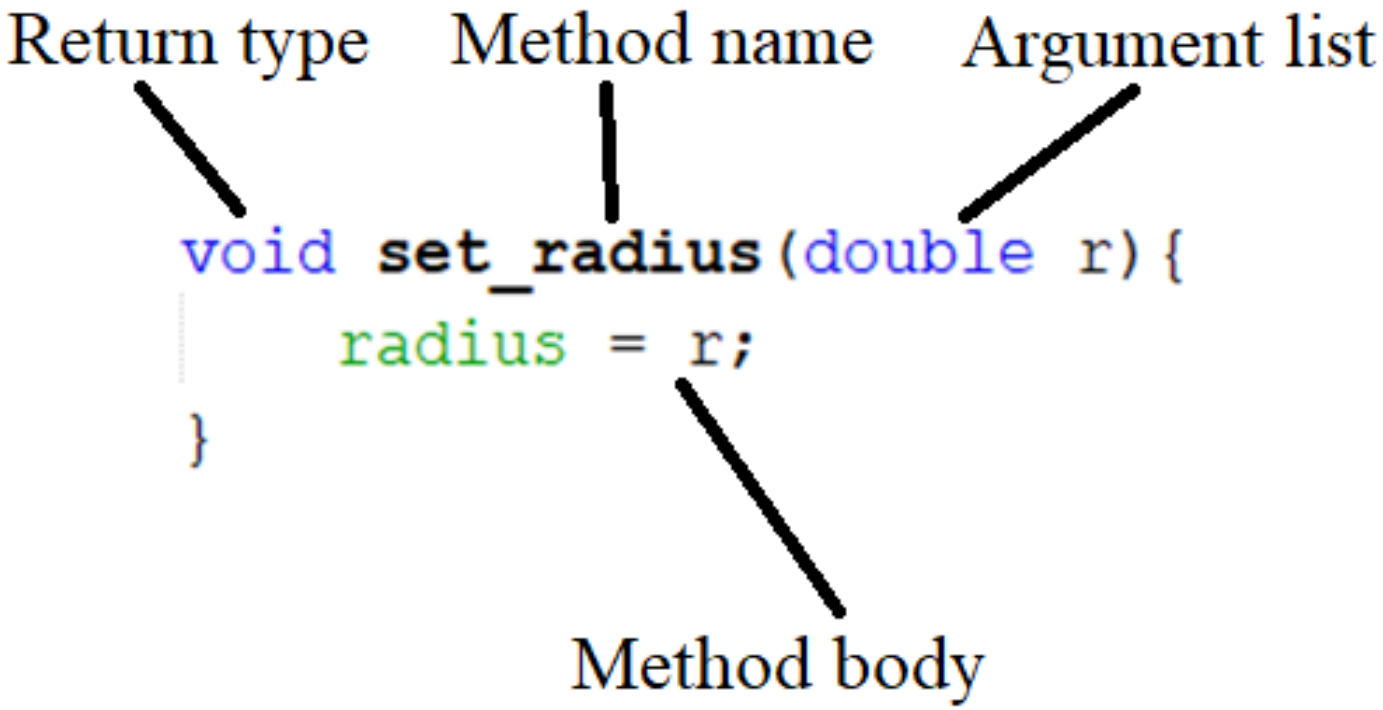
- We can add methods by specifying the following
- 1. Access specifier (public/private/default etc)
- 2. Return type of the methods
- 3. Method name
- 4. Argument list
- 5. Method body
- 6. Other eg static, final etc.

Adding Methods

- The class circle defines two methods set_radius() and display_circle_area

```
void set_radius(double r){
    radius = r;
}
void display_circle_area(){
    area = 3.14*radius*radius;
    System.out.print("\n Radius =" + radius);
    System.out.println("\n Area =" + area);
}
```

Adding Methods



- Here access specifier is not specified so it have default access.

Parameterized methods

- Method definition consists of a method header and a method body.
- The same is shown in the following syntax –
- modifier returnType nameOfMethod (Parameter List)

```
{
  // method body
}
```

- Methods with non empty Parameter List are called parameterized methods. Example

```
void set_radius(double r){
    radius = r;
}
```

Creating Objects

- In Java, an object is created from a class. We can create objects by using:

class-name, object-reference , = operator, new keyword and a constructor of the class

- For example : The first line is creating an object of type Circle.

```
Circle c1 = new Circle();
c1.set_radius(2.0);
c1.display_circle_area();
```

Constructor

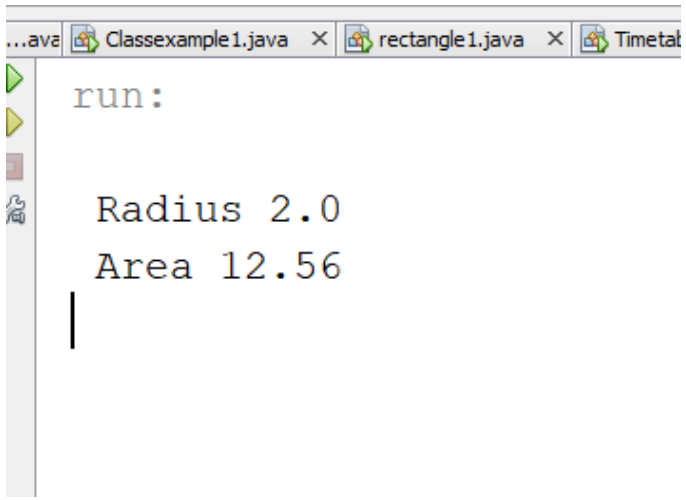
- Constructors are used to initialize the object's state/data members.
- A constructor also contains collection of statements that are executed at time of Object creation.
- So constructors are used to assign values to the class variables at the time of object creation, either explicitly done by the programmer or by Java itself (default constructor).

Constructor Example

```
class circle
{
    double radius,area;
    circle()
    {
        radius=0;
    }
}
```

```
class circle
{
    double radius,area;
    circle()
    {
        radius=0;
    }
    circle(double r)
    {
        radius=r;
    }
    void calarea()
    {
        area = 3.14 *radius*radius;
        System.out.print("\n Radius " + radius);
        System.out.print("\n Area " + area);
    }
}
```

```
class circle
{
    double radius,area;
    circle()
    {
        radius=0;
    }
    circle(double r)
    {
        radius=r;
    }
    void calarea(){
        area = 3.14 *radius*radius;
        System.out.print("\n Radius " + radius);
        System.out.print("\n Area " + area);
    }
}
public class constructor1
{
    public static void main(String args[]){
        circle c1 = new circle(2);
        c1.calarea();
    }
}
```



When is a Constructor called ?

- A constructor is invoked at the time of object or instance creation.
- Each time an object is created using new() keyword at least one constructor is invoked to assign initial values to the data members of the same class.

Rules for writing Constructor:

- Constructor(s) of a class must has same name as the class name in which it resides.
- A constructor in Java can not be abstract, final, static and Synchronized.
- Access modifiers can be used in constructor declaration to control its access *i.e* which other class can call the constructor.

How constructors are different from methods in Java?

- Constructor(s) must have the same name as the class within which it defined while it is not necessary for the method in java.
- Constructor(s) do not return any type while method(s) have the return type or **void** if does not return any value.
- Constructor is called only once at the time of Object creation while method(s) can be called any numbers of time.

Types of constructor

- **No-argument constructor:**
- **Parameterized Constructor**
- **Constructor Overloading**

No-argument constructor: default constructor

- A constructor that has no parameter is known as default constructor. If we don’t define a constructor in a class, then compiler creates **default constructor(with no arguments)** for the class.
- If we write a constructor with arguments or no-arguments then the compiler does not create a default constructor.
- Example

No-argument constructor: default constructor

```
class circle
{
    double radius,area;
    circle()
    {
        radius=0;
    }
}
```

Parameterized Constructor:

- A constructor that has parameters is known as parameterized constructor.
- If we want to initialize fields of the class with your own values, then use a parameterized constructor.

Parameterized Constructor:

```
class circle
{
    double radius,area;
    circle(double r)
    {
        radius=r;
    }
}
```

CONSTRUCTOR VERSUS METHOD

Constructor	Method
Constructors are special type of methods used to initialize objects of its class.	Methods are a set of instructions that are invoked at any point in a program to return a result.
The purpose of a constructor is to create an instance of a class.	The purpose of a method is to execute Java code.
They are called implicitly immediately upon object creation.	They can be called directly without even creating an instance of that class.
They are used to initialize objects that don't exist.	They perform operations on already created objects.
The name of the constructor must be same as the name of the class.	They can have arbitrary names in Java.
They are not inherited by subclasses.	They are inherited by subclasses.

Exercise : class Rectangle

- Create a class Rectangle and add following
- Data Members
    1. Length, Breadth and Area
- Methods
    1. Calculate area
    2. Display
- Constructor
    1. Default
    2. Parameterized

Exercise : class BankAccount

- Create a class BankAccount and add following
- Data Members
    1. Name as string
    2. Balance as double
- Methods
    1. Deposit
    2. Withdraw
    3. Display
- Constructor
    1. Default
    2. Parameterised

Some Examples

- Create class for
- Rectangle
- Bank Account

The this Keyword

There can be a lot of usage of java **this** keyword. In java, this is a **reference variable** that refers to the current object.

The this Keyword

- In the body of a method definition, sometimes you need to refer to the object that contains the method (*in other words, the object itself*).
- To refer to the object in its own method, use the this keyword where you normally would refer to an object's name.
- The this keyword refers to the current object which is calling that method.
- Because this is a reference to the current instance of a class, only use it inside the body of an instance method definition.

Usage of Java this Keyword

There can be a lot of usage of java this keyword. In java, this is a reference variable that refers to the current object.

01 this can be used to refer current class instance variable.

02 this can be used to invoke current class method (implicity)

03 this() can be used to invoke current class Constructor.

04 this can be passed as an argument in the method call.

05 this can be passed as argument in the constructor call.

06 this can be used to return the current class instance from the method

1) this: to refer current class instance variable

The this keyword can be used to refer current class instance variable. If there is ambiguity between the instance variables and parameters, this keyword resolves the problem of ambiguity.

```
class Student{
int rollno;
String name;
float fee;
Student(int rollno,String name,float fee){
rollno=rollno;
name=name;
fee=fee;
}
void display(){System.out.println(rollno+" "+name+" "+fee);}
}
class TestThis1{
public static void main(String args[]){
Student s1=new Student(111,"ankit",5000f);
Student s2=new Student(112,"sumit",6000f);
s1.display();
s2.display();
}}
```

Instance Variables

Parameters/local variables

Output:

```
0 null 0.0
0 null 0.0
```

In the above example, parameters (formal arguments) and instance variables are same. So, we are using this keyword to distinguish local variable and instance variable.

Solution of the preceding problem by this keyword

```
class Student{
int rollno;
String name;
float fee;
Student(int rollno,String name,float fee){
this.rollno=rollno;
this.name=name;
this.fee=fee;
}
void display(){System.out.println(rollno+" "+name+" "+fee);}
}

class TestThis2{
public static void main(String args[]){
Student s1=new Student(111,"ankit",5000f);
Student s2=new Student(112,"sumit",6000f);
s1.display();
s2.display();
}}
```

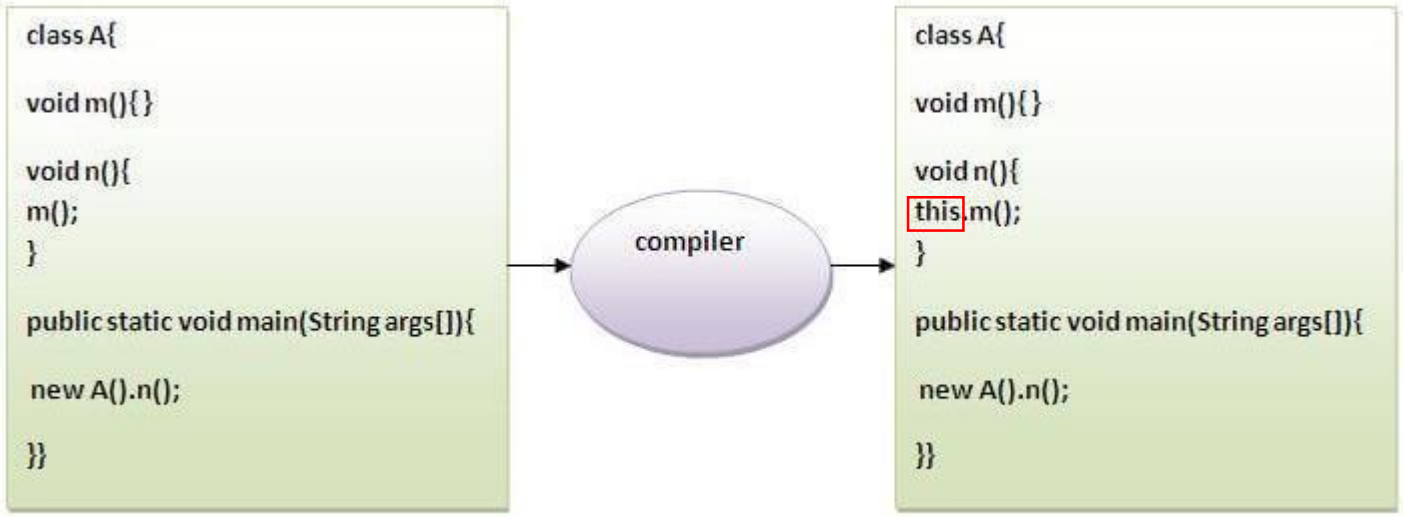
Output:

```
111 ankit 5000.0
112 sumit 6000.0
```

Note: If local variables (formal arguments) and instance variables are different, there is no need to use this keyword.

2) this: to invoke current class method.

We may invoke the method of the current class by using the this keyword. If you don't use the this keyword, compiler automatically adds this keyword while invoking the method.




```
class A{
void m(){System.out.println("hello m");}
void n(){
System.out.println("hello n");
//m();//same as this.m()
this.m();
}
}
class TestThis4{
public static void main(String args[]){
A a=new A();
a.n();
}}
```

Output

```
hello n
hello m
```

3) this() : to invoke current class constructor

The this() constructor call can be used to invoke the current class constructor. It is used to reuse the constructor. In other words, it is used for constructor chaining.

Calling default constructor from parameterized constructor:

```
class A{
A(){System.out.println("hello a");}
A(int x){
this();
System.out.println(x);
}
}
class TestThis5{
public static void main(String args[]){
A a=new A(10);
}}
```

Default Constructor

Parametrized Constructor

Output:

```
hello a
10
```

3) this() : to invoke current class constructor

Calling parameterized constructor from default constructor:

```
class A{
A(){
this(5);
System.out.println("hello a");
}
A(int x){
System.out.println(x);
}
}
class TestThis6{
public static void main(String args[]){
A a=new A();
}}
```

Default Constructor

Parametrized Constructor

Output

```
5
hello a
```

4) this: to pass as an argument in the method

The this keyword can also be passed as an argument in the method. It is mainly used in the event handling.

```
class S2{
  void m(S2 obj){
  System.out.println("method is invoked");
  }
  void p(){
  m(this);
  }
  public static void main(String args[]){
  S2 s1 = new S2();
  s1.p();
  }
}
```

Output:

```
method is invoked
```

Application of this that can be passed as an argument:

In event handling (or) in a situation where we have to *provide reference of a class to another one*. It is used to reuse one object in many methods.

5) this: to pass as argument in the constructor call

We can pass the this keyword in the constructor also. It is useful if we have to use one object in multiple classes.

```
class B{
 A4 obj;
 B(A4 obj){
   this.obj=obj;
 }
 void display(){
   System.out.println(obj.data);//using data member of A4 class
 }
}
```

Step 3

```
class A4{
 int data=10;
 A4(){
  B b=new B(this);
  b.display();
 }
```

Step 2

```
 public static void main(String args[]){
  A4 a=new A4();
 }
}
```

Step 1

```
Output:10
```

6) this keyword can be used to return current class instance

We can return this keyword as an statement from the method. In such case, return type of the method must be the class type (non-primitive).

Syntax of this that can be returned as a statement

```
return_type method_name(){
return this;
}
```

Example of this keyword that you return as a statement from the method

```
class A{
A getA(){
return this;
}
void msg(){System.out.println("Hello java");}
}
class Test1{
public static void main(String args[]){
new A().getA().msg();
}
}
```

Output:

```
Hello java
```

Proving this keyword

Let's prove that this keyword refers to the current class instance variable. In this program, we are printing the reference variable and this, output of both variables are same.

```
class A5{
void m(){
System.out.println(this);//prints same reference ID
}
public static void main(String args[]){
A5 obj=new A5();
System.out.println(obj);//prints the reference ID
obj.m();
}
}
```

Output:

```
A5@22b3ea59
A5@22b3ea59
```

Java Variables

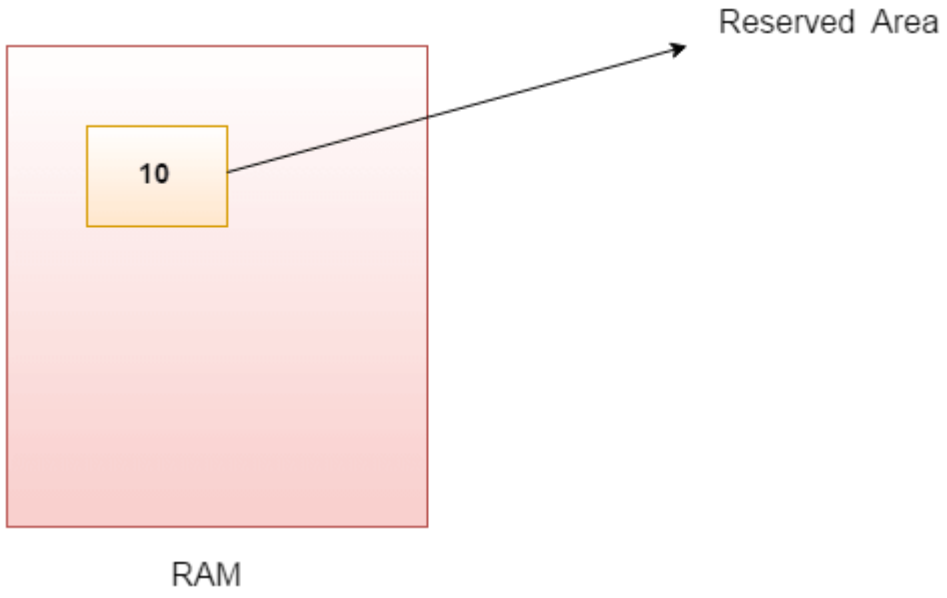
A variable is a container which holds the value while the Java program is executed. A variable is assigned with a data type.

Variable is a name of memory location. There are three types of variables in java: **local, instance** and **static**.

There are two types of data types in Java: **primitive** and **non-primitive**.

Java Variables

A variable is the name of a reserved area allocated in memory. In other words, it is a name of the memory location. It is a combination of "vary + able" which means its value can be changed.



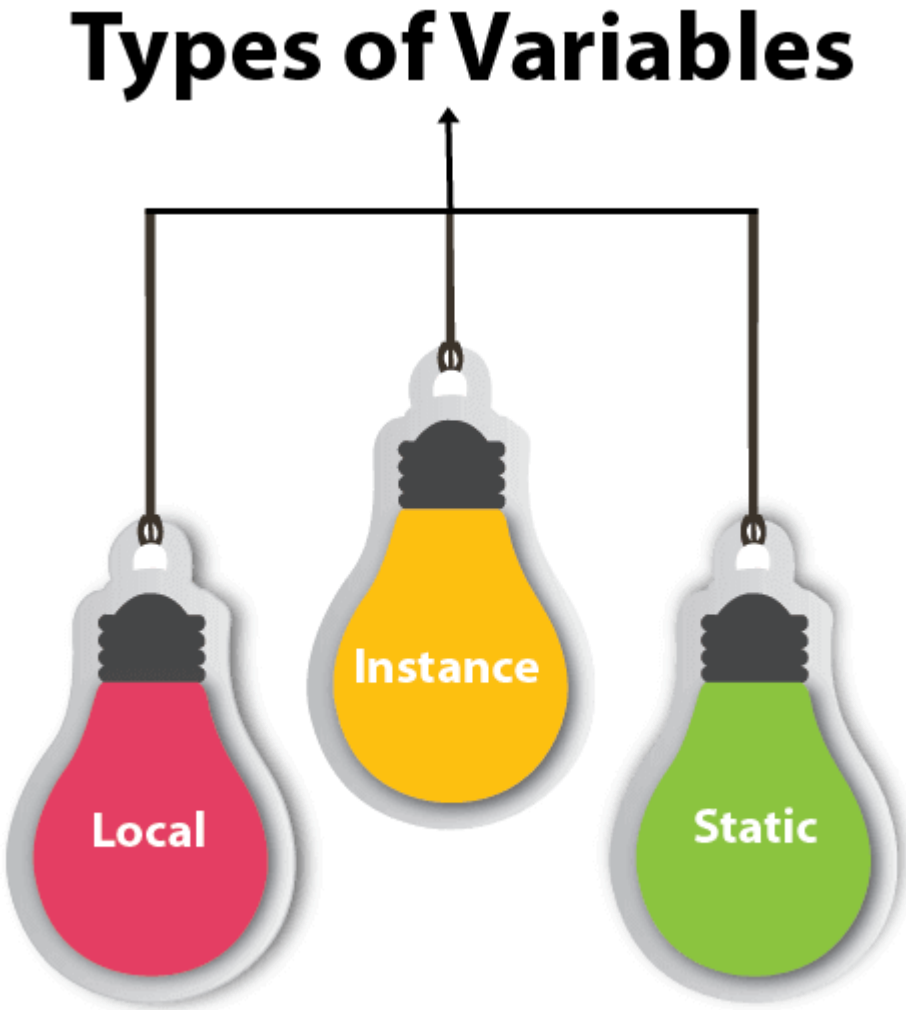

```
int data=50;//Here data is variable
```

Types of Variables

There are three types of variables

in Java:

1. Local variable
2. Instance variable
3. Static variable



1) Local Variable

A variable declared inside the body of the method is called local variable. You can use this variable only within that method and the other methods in the class aren't even aware that the variable exists.

A local variable cannot be defined with "static" keyword.

2) Instance Variable

A variable declared inside the class but outside the body of the method, is called an instance variable. It is not declared as static.

It is called an instance variable because its value is instance-specific and is not shared among instances.

3) Static variable

A variable that is declared as static is called a static variable. **It cannot be local.**

You can create a single copy of the static variable and share it among all the instances of the class.

Memory allocation for static variables happens **only once** when the class is loaded in the memory.

Example to understand the types of variables in java

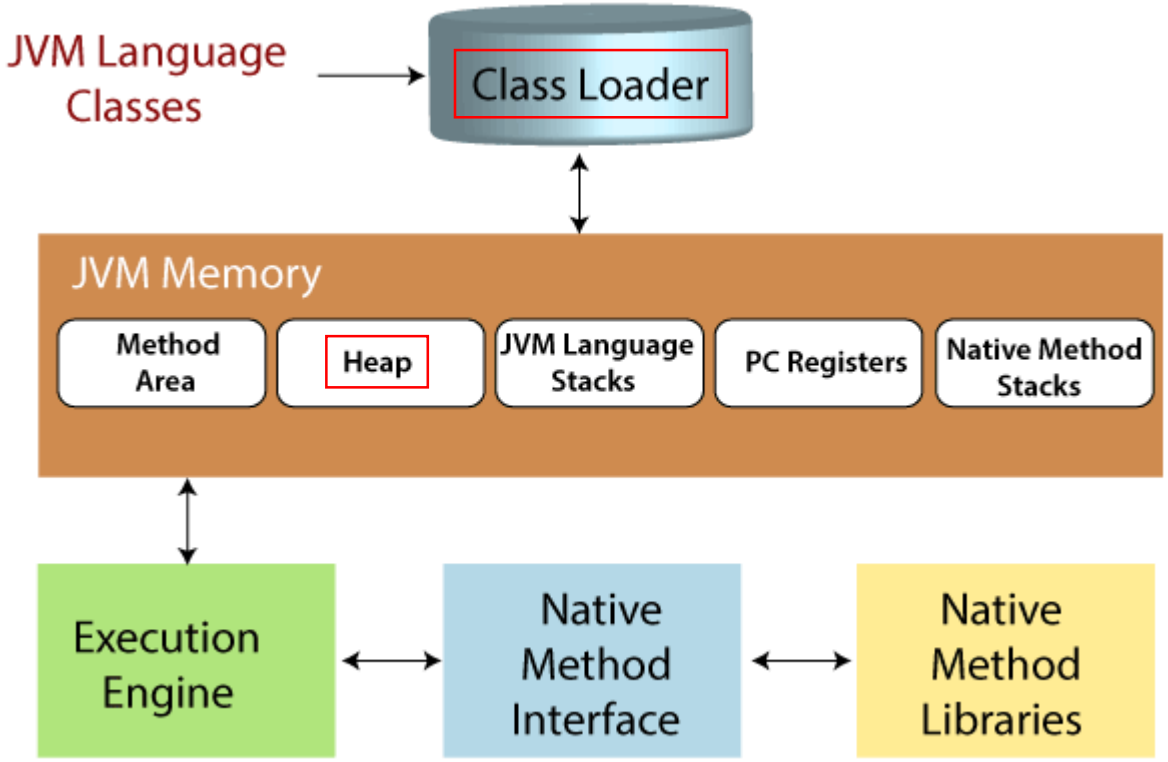
```
public class A
{
   static int m=100;//static variable
   void method()
   {
      int n=90;//local variable
   }
   public static void main(String args[])
   {
      int data=50;//instance variable
   }
}//end of class
```

Static Keyword

- The static keyword in Java is used for memory management mainly.
- We can *apply* static keyword with **variables** or **methods** .
- The static keyword *belongs* to the **whole class** but **not** to an **instance of the class**.

Static variables are stored inside Class Loader Memory.

Instance variables are stored inside Heap memory.



Static variables

- The static variable can be used to refer to the **common property of all objects** (*which is not unique for each object*), for example, the company name of employees, college name of students, etc.
- The static variable *gets* memory **only once** in the **class-loader memory** at the time of class loading.
- It makes your program **memory efficient** (*i.e.,* it saves memory).

Understanding the problem without static variable

```
class Student
{
    int rollno;
    String name;
    String college=“GLA";
}
```

- Suppose there are 500 students in my college, now all instance data members will get memory each time when the object is created.
- All students have its unique rollno and name, so instance data member is good in such case.
- Here, "college" refers to the common property of all objects. If we make it static, this field will get the memory only once.

Java Program to demonstrate the use of static variable


```
//Java Program to demonstrate the use of static variable
class Student{
  int rollno;              //instance/ Non-static variable
  String name;             //instance/ Non-static variable
  static String college ="GLA";          //static variable
  //constructor
  Student(int r, String n){
  rollno = r;
  name = n;
  }
  //method to display the values
  void display (){
  System.out.println(rollno +" "+ name + " “ + college);  }
}
//Test class to show the values of objects
public class TestStaticVariable1{
 public static void main(String args[]){
 Student s1 = new Student(111,"Karan");
 Student s2 = new Student(222,"Aryan");
 //we can change the college of all objects by the single line of code
 //Student.college="Galgotia";
 s1.display();
 s2.display();
 }
}
```

```java
//Java Program to demonstrate the use of static variable
class Student{
    int rollno;//instance variable
    String name;
    static String college ="GLA";//static variable
    //constructor
    Student(int r, String n){
    rollno = r;
    name = n;
    }
    //method to display the values
    void display (){
        System.out.println(rollno+" "+name+" "+college);
    }
}
```

```java
//Java Program to demonstrate the use of static variable
class Student{
    int rollno;//instance variable
    String name;
    static String college ="GLA";//static variable
    //constructor
    Student(int r, String n){
    rollno = r;
    name = n;
    }
    //method to display the values
    void display (){
        System.out.println(rollno+" "+name+" "+college);
    }
}
//Test class to show the values of objects
public class TestStaticVariable1{
 public static void main(String args[]){
  Student s1 = new Student(111,"Karan");
  Student s2 = new Student(222,"Aryan");
  //we can change the college of all objects by the single line of code
  //Student.college="Galgotia";
  s1.display();
  s2.display();
 }
}
```

Static Variable

```
run:
111 Karan GLA
222 Aryan GLA
BUILD SUCCESSFUL (
```

Java static method

- If you apply **static** keyword with any method, it is known as static method.
- A static method *belongs* to the **class** rather than the object of a class.
- A static method can be *invoked* without the need for creating an instance (object) of a class.
- **A static method can *access* static data member and can *change* the value of it.**

Java Program to demonstrate the use of a static method.

```
//Java Program to demonstrate the use of a static method.
class Student{
int rollno;
String name;
static String college = "GLA";

//static method to change the value of static variable
static void change(String college2)
{ college = college2; }

 //constructor to initialize the variable
Student(int r, String n)
{ rollno = r;
name = n; }

//method to display values
void display()
{System.out.println
(rollno + " "+name+" "+college);}
}

//Test class to create and display the values of object
public class TestStaticMethod{
public static void main(String args[])
{

//creating objects
Student s1 = new Student(111,"Karan");
Student s2 = new Student(222,"Aryan");

Student.change("GLA University");

//calling change method
Student s3 = new Student(333,"Sonoo");

 //calling display method
 s1.display();
s2.display();
s3.display();
}
}
```

```java
//Java Program to demonstrate the use of a static method.
class Student{
      int rollno;
      String name;
      static String college = "GLA";
      //static method to change the value of static variable
      static void change(String college2){
      college = college2;
      }
      //constructor to initialize the variable
      Student(int r, String n){
      rollno = r;
      name = n;
      }
      //method to display values
      void display(){System.out.println(rollno+" "+name+" "+college);}
}
```

```java
//Test class to create and display the values of object
public class TestStaticMethod{
    public static void main(String args[])
    {
    //creating objects
    Student s1 = new Student(111,"Karan");
    Student s2 = new Student(222,"Aryan");
    Student.change("GLA University");//calling change method
    Student s3 = new Student(333,"Sonoo");
    //calling display method
    s1.display();
    s2.display();
    s3.display();
    }
}
```

```
run:
111 Karan GLA University
222 Aryan GLA University
333 Sonoo GLA University
BUILD SUCCESSFUL (total time: 0 seconds)
```

Another example


```
class Calculate
{
  static int cube(int x)
  {
    return x*x*x;
  }
  static int square(int x)
  {
      return x*x;
  }
  public static void main(String args[])
  {
  int result=Calculate.cube(5);
  System.out.println(result);
  result=Calculate.square(9);
  System.out.println(result);
  }
}
```

```
run:
125
81
BUILD SUCCESSFUL (tot
```

Another example

```
class X
{
   int x1;
   static int y1=1200;
   static void change_y1(int a)
   {
      y1=a;
   }
   void change_x1(int b)
   {
      x1=b;
   }
   void display()
   {
      System.out.print("\n x1 = " + x1);
      System.out.print("\n y1 = " + y1);
   }
}
public class staticVariables
{
   public static void main(String args[])
   {
      X obj1 = new X();
      obj1.change_x1(100);
      System.out.print("\n Object 1");
      obj1.display();
      X obj2 = new X();
      obj2.change_x1(200);
      X.change_y1(1500);
      System.out.print("\n Object 2");
      obj2.display();

      System.out.print("\n Object 1");
      obj1.display();
   }
}
```

Restrictions for the static method

There are two main restrictions for the static method.

They are:

1. The static method **can not** use *non static data member* or *call non-static method directly.*
2. **this** and **super** cannot be used in static context.